

优化模型

主讲：复旦大学数学科学学院 曹沅

目录

1 引言：数学规划的基本框架	2
2 线性规划 (Linear Programming, LP)	2
2.1 标准形式	2
2.2 例：营养问题	2
2.3 单纯形法的基本思想	4
2.4 实例	4
3 整数规划 (Integer Programming, IP)	6
3.1 例：0-1 背包问题	6
3.2 实例	7
3.3 实例	8
3.4 例：分配问题 (Assignment Problem / 指派问题)	9
3.5 实例	10
3.6 例：装载问题	12
3.7 分支定界法的基本思想	12
3.8 实例：IP 问题	13
3.9 例：选址问题	14
3.10 例：旅行售货员问题 (Travelling Salesman Problem, TSP)	16
4 非线性规划 (Nonlinear Programming, NLP)	19
4.1 一般形式与最优性条件	19
4.2 二次规划 (Quadratic Programming, QP)	20

1 引言：数学规划的基本框架

一个数学规划（优化模型）问题一般由三部分组成：

- **决策变量**（decision variables）：需要求解的未知量，记为 $\vec{x} = (x_1, x_2, \dots, x_n)^T$ ；
- **目标函数**（objective function）：衡量方案优劣的函数 $f(\vec{x})$ ，问题是求它的最大值或最小值；
- **约束条件**（constraints）：决策变量必须满足的等式或不等式关系。

满足全部约束条件的 $\vec{x}$ 称为**可行解**，全体可行解构成**可行域**，使目标函数取到最优值的可行解称为**最优解**，对应的目标函数值称为**最优值**。

按目标函数、约束条件的性质，可以对数学规划分类：

分类依据	类型
目标/约束是否线性	线性规划（LP）/ 非线性规划（NLP）
决策变量是否要求整数	整数规划（IP，含 0-1 规划）/ 连续规划
目标函数是否二次、约束是否线性	二次规划（QP）
是否混有连续变量与整数变量	混合整数规划（MIP）

2 线性规划（Linear Programming, LP）

2.1 标准形式

任何线性规划都可以通过下述等价变换化为**标准形**（目标求最小、约束为等式、变量非负）：

- $\max f = \min(-f)$ ；
- 不等式 $\sum a_j x_j \leq b$ 可引入松弛变量 $s \geq 0$ 化为 $\sum a_j x_j + s = b$ ； $\sum a_j x_j \geq b$ 可引入剩余变量 $s \geq 0$ 化为 $\sum a_j x_j - s = b$ ；
- 自由变量 x_j （无符号限制）可写成两个非负变量之差 $x_j = x_j^+ - x_j^-$, $x_j^+, x_j^- \geq 0$ 。这一步是后续单纯形法能够统一处理各种线性规划的基础。

2.2 例：营养问题

一个养鸡场用 n 种饲料配制一批配合饲料。

要求这批配合饲料中必须含有 m 种不同的营养成分，且要求 $i^{\#}$ 营养成分的含量不低于 b_i ；

已知每单位（克或千克）的 $j^{\#}$ 饲料中含有 $i^{\#}$ 营养成分 a_{ij} ，每单位 $j^{\#}$ 饲料的价格为 c_j 。在保证营养的前提下，问采用怎样的配方能使这批配合饲料的费用最小？

设 x_j 为 $j^{\#}$ 饲料的用量，由已知条件可得下表（行是 m 种营养成分及其下限，列是 n 种饲料）：

营养成分\饲料	1 [#]	2 [#]	...	$n^{\#}$
1 [#] ($\geq b_1$)	a_{11}	a_{12}	$\cdots$	a_{1n}
2 [#] ($\geq b_2$)	a_{21}	a_{22}	$\cdots$	a_{2n}
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
$m^{\#}$ ($\geq b_m$)	a_{m1}	a_{m2}	$\cdots$	a_{mn}
单价	c_1	c_2	$\cdots$	c_n

(1)

则 $\sum_{j=1}^n a_{ij}x_j$ ($i = 1, 2, \dots, m$) 就是这批配合饲料中 $i^{\#}$ 营养成分的总含量，按题意

$$\sum_{j=1}^n a_{ij}x_j \geq b_i, \quad i = 1, 2, \dots, m, \quad (2)$$

这批配合饲料的总费用是 $\sum_{j=1}^n c_jx_j$ ，于是问题归结为

$$\begin{aligned} \min \quad & f = \sum_{j=1}^n c_jx_j \\ \text{s.t.} \quad & \sum_{j=1}^n a_{ij}x_j \geq b_i, \quad i = 1, 2, \dots, m \\ & x_j \geq 0, \quad j = 1, 2, \dots, n. \end{aligned} \quad (3)$$

向量、矩阵写法：

$$\begin{aligned} \min \quad & f = \vec{c}^T \vec{x}, \\ \text{s.t.} \quad & A\vec{x} \geq \vec{b}, \\ & \vec{x} \geq \vec{0}, \end{aligned} \quad (4)$$

其中

$$\vec{c} = \begin{pmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{pmatrix}, \quad \vec{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}, \quad \vec{b} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix}, \quad A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}. \quad (5)$$

x_j ——决策变量， f ——目标函数，不等式（或等式）——约束条件；若目标函数、约束条件关于决策变量都是线性的，就称 (3) 是线性规划问题（Linear Programming, LP）。

2.3 单纯形法的基本思想

- 可行域 $\{\vec{x} : A\vec{x} \geq \vec{b}, \vec{x} \geq \vec{0}\}$ 是凸多面体，若最优值有限，则一定可以在某个顶点（基本可行解）处取到。
- 单纯形法的思路是：从一个顶点出发，沿着使目标函数下降（或上升）的棱不断转移到相邻顶点，直到找不到能继续改进的方向为止，此时即为最优顶点。
- 每一步“转移顶点”对应一次**换基**（选择换入变量与换出变量），当问题存在退化基本可行解时，简单的换基规则可能出现基反复循环、算法不终止的情况；理论上可以证明，采用 **Bland 规则**（在换入/换出变量有多个候选时，恒选下标最小者）的单纯形法**不会出现死循环**，从而保证算法有限步终止。
- 在 Matlab 的优化工具箱中，`linprog` 函数封装了求解线性规划的数值方法（内点法或单纯形法），可以直接调用，不必手工迭代。

`linprog` 函数的调用格式是：

```
[x, fval] = linprog(f, A, b, Aeq, beq, lb, ub)
```

其中 f 是目标函数系数向量（默认求 $\min$ ）， A, b 对应不等式约束 $Ax \leq b$ ， Aeq, beq 对应等式约束， lb, ub 是变量的上下界；若无某类约束，用 $[]$ 占位。

2.4 实例

$$\begin{aligned}
 \min \quad & f = -6x_1 - 5x_2 \\
 \text{s.t.} \quad & 5x_1 + 3x_2 \leq 100 \\
 & 6x_1 + 7x_2 \leq 160 \\
 & 5x_2 \leq 180 \\
 & x_1, x_2 \geq 0
 \end{aligned} \tag{6}$$

改造成矩阵向量形式：

$$\begin{aligned}
 \min \quad & f = (-6, -5) \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \\
 \text{s.t.} \quad & \begin{pmatrix} 5 & 3 \\ 6 & 7 \\ 0 & 5 \\ -1 & 0 \\ 0 & -1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \leq \begin{pmatrix} 100 \\ 160 \\ 180 \\ 0 \\ 0 \end{pmatrix}
 \end{aligned} \tag{7}$$

Matlab 代码：

```
clear
f = [-6, -5];
A = [5 3; 6 7; 0 5; -1 0; 0 -1];
b = [100, 160, 180, 0, 0];
[x, fval] = linprog(f, A, b)
```

计算结果：

```
Optimization terminated.
x =
    12.9412
    11.7647
fval =
   -136.4706
```

所以最优解是 (12.9412, 11.7647)；最优值是 $f = -136.4706$ 。

手工验证（图解法/顶点枚举）：由于只有两个变量，可行域是一个凸多边形，最优解必在顶点处取得，逐一列出顶点并代入目标函数：

顶点来源	顶点坐标	$f = -6x_1 - 5x_2$
$x_1 = x_2 = 0$	(0, 0)	0
$x_2 = 0$, $5x_1 = 100$ （第一条起作用，比 $6x_1 \leq 160$ 更紧）	(20, 0)	-120
$x_1 = 0$, 取三条约束中最紧的 $7x_2 \leq 160$	(0, 22.857)	-114.29
第一、二条约束交点： $5x_1 + 3x_2 = 100$, $6x_1 + 7x_2 = 160$	(12.9412, 11.7647)	-136.47

解交点方程：由 $5x_1 + 3x_2 = 100$ 乘 7、 $6x_1 + 7x_2 = 160$ 乘 3 相减，得 $17x_1 = 220 \Rightarrow x_1 = 12.9412$ ，代回得 $x_2 = 11.7647$ 。四个顶点中 f 的最小值在 (12.9412, 11.7647) 处取得，与 linprog 结果一致。

另一种调用方式（显式列出各类约束）：

```
clear
f = [-6, -5];
A = [5 3; 6 7; 0 5];
b = [100, 160, 180];
lb = [0, 0]; % 自变量下界
ub = []; % 自变量上界
Aeq = []; % 等式约束条件系数矩阵
```

```

beq = [];           % 等式约束条件系数向量
[x, fval] = linprog(f, A, b, Aeq, beq, lb, ub)

% Optimization terminated.
% x =
%    12.9412
%    11.7647
% fval =
%   -136.4706

```

3 整数规划 (Integer Programming, IP)

相关竞赛题：2003 年 CUMCM B 题（露天矿生产的车辆安排），2005 年 CUMCM B 题（DVD 在线租赁问题）。

有些实际问题中决策变量部分或全部只能取整数，如人数、项目数、工厂数等，这类问题称为**整数规划**（IP）。更加特别地，若决策变量只能取 0 或 1，则称为 **0-1 规划**（Binary Integer Programming, BIP）。整数规划的求解本质上比线性规划困难得多（一般是 NP-难问题），常见方法有分支定界法、割平面法、动态规划、以及针对特殊结构（如指派问题）的专门算法。

3.1 例：0-1 背包问题

设一个背包的容量为 b ，要从 n 件物品中选择若干件装入背包，设 $j^{\#}$ 物品的体积为 a_j ，价值为 c_j ，问应该将哪些物品装入背包可使总价值最大？

记决策变量

$$x_j = \begin{cases} 1 & \text{将 } j^{\#} \text{ 物品装入背包} \\ 0 & \text{否则} \end{cases} \quad (8)$$

则总价值（目标函数）为： $\sum_{j=1}^n c_j x_j$,

总体积（约束条件）为： $\sum_{j=1}^n a_j x_j$,

所以 0-1 背包问题就是：

$$\begin{aligned}
 \max \quad & z = \sum_{j=1}^n c_j x_j \\
 \text{s.t.} \quad & \sum_{j=1}^n a_j x_j \leq b \\
 & x_j = 0, 1, \quad j = 1, 2, \dots, n.
 \end{aligned} \tag{9}$$

这是最简单的 0-1 规划问题，是很多整数规划、0-1 规划的基础。它可以用回溯法、分支定界法、动态规划等方法精确求解，也可以用贪心算法（按 c_j/a_j 单位价值排序装填）求近似解。经典的动态规划递推为：设 $V(k, w)$ 为只考虑前 k 件物品、容量为 w 时能得到的最大价值，则

$$V(k, w) = \begin{cases} V(k-1, w), & a_k > w \\ \max\{V(k-1, w), V(k-1, w-a_k) + c_k\}, & a_k \leq w \end{cases} \tag{10}$$

Matlab 中有一个函数 `bintprog` 可以求解 0-1 规划问题（Binary Integer Programming），调用格式是：

```
[x, fval] = bintprog(f, A, b, Aeq, beq)
```

说明：`bintprog` 是较早版本 Matlab 优化工具箱中的函数，在较新版本中已被移除，官方推荐改用能处理混合整数线性规划的 `intlinprog`，调用格式为

```
[x, fval] = intlinprog(f, intcon, A, b, Aeq, beq, lb, ub)
```

其中 `intcon` 是取整数（0-1 变量取 `lb=0, ub=1` 且整数）的变量下标。下文仍按原讲义使用 `bintprog` 演示建模思路，读者在新版 Matlab 中可等价替换为 `intlinprog`。

3.2 实例

设 $b = 55$, $(a_j) = (9, 11, 13, 12, 10, 12)$, $(c_j) = (5, 4, 5, 6, 7, 4)$ 。

问题（转化为求 $\min$ ）：

$$\begin{aligned}
 \min \quad & z = - \sum_{j=1}^6 c_j x_j \\
 \text{s.t.} \quad & \sum_{j=1}^6 a_j x_j \leq b \\
 & x_j = 0, 1, \quad j = 1, 2, \dots, 6.
 \end{aligned} \tag{11}$$

```
clear
f = [-5, -4, -5, -6, -7, -4];
```

```
A = [9, 11, 13, 12, 10, 12];
b = 55;
[y, fval] = bintprog(f, A, b, [], [])

% Optimization terminated.
% y =
%      1      1      1      1      1      0
% fval =
%     -27
```

所以最优解是： $x_1 = 1, x_2 = 1, x_3 = 1, x_4 = 1, x_5 = 1, x_6 = 0$ ，此时总体积 $= 9 + 11 + 13 + 12 + 10 = 55 = b$ （恰好装满），原问题（求 max）的最优值是 27。

3.3 实例

$$\begin{aligned}
 \min \quad & (-3x_1 + 2x_2 - 5x_3) \\
 \text{s.t.} \quad & x_1 + 2x_2 - x_3 \leq 2 \\
 & x_1 + 4x_2 + x_3 \leq 4 \\
 & x_1 + x_2 \leq 5 \\
 & 4x_2 + x_3 \leq 6 \\
 & x_i = 0, 1, \quad i = 1, 2, 3
 \end{aligned} \tag{12}$$

```
clear
f = [-3, 2, -5];
A = [1 2 -1; 1 4 1; 1 1 0; 0 4 1];
b = [2; 4; 5; 6];
[x, fval] = bintprog(f, A, b, [], [])

% Optimization terminated.
% x =
%      1      0      1
% fval =
%     -8
```

所以最优解为 $x_1 = 1, x_2 = 0, x_3 = 1$ ，最优值为 $f = -8$ 。校验： $x_1 + 2x_2 - x_3 = 1 - 1 = 0 \leq 2$ ； $x_1 + 4x_2 + x_3 = 1 + 0 + 1 = 2 \leq 4$ ； $x_1 + x_2 = 1 \leq 5$ ； $4x_2 + x_3 = 1 \leq 6$ ，全部约束满足。

3.4 例：分配问题（Assignment Problem / 指派问题）

给 n 个人分配 n 项工作，每人一项工作；已知 $i^{\#}$ 人完成 $j^{\#}$ 工作需时间（或费用） c_{ij} ，问怎样分配使得完成全部工作的总时间（或费用）最少？

建模：设决策变量是

$$x_{ij} = \begin{cases} 1 & \text{分配 } i^{\#} \text{ 人去完成 } j^{\#} \text{ 工作} \\ 0 & \text{否则} \end{cases} \quad (13)$$

则花费总时间（目标函数）： $\sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij}$ 。

对 $i^{\#}$ 人来说，只分配给一项工作（约束条件）：

$$\sum_{j=1}^n x_{ij} = 1, \quad i = 1, 2, \dots, n. \quad (14)$$

对 $j^{\#}$ 工作来说，只分配给一个人（约束条件）：

$$\sum_{i=1}^n x_{ij} = 1, \quad j = 1, 2, \dots, n. \quad (15)$$

所以分配问题就是 **0-1 规划**：

$$\begin{aligned} \min \quad & z = \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_{j=1}^n x_{ij} = 1, \quad i = 1, 2, \dots, n \\ & \sum_{i=1}^n x_{ij} = 1, \quad j = 1, 2, \dots, n \\ & x_{ij} = 0, 1, \quad i = 1, 2, \dots, n, j = 1, 2, \dots, n. \end{aligned} \quad (16)$$

匈牙利算法要点（专用于分配问题，比通用整数规划算法高效得多，多项式时间 $O(n^3)$ ）：

1. 对费用矩阵 $C = (c_{ij})$ ，每行减去该行最小元素，再每列减去该列最小元素，得到含 0 元素的矩阵；
2. 用最少数目的横线、竖线覆盖矩阵中全部 0 元素；若覆盖直线数等于 n ，则可以从 0 元素中选出一组「独立 0 元素」（每行每列恰有一个），对应的指派即为最优解；
3. 若覆盖直线数小于 n ，则在未覆盖元素中找最小值 δ ，未被覆盖的行都减去 δ ，被覆盖的列都加上 δ ，回到第 2 步。

由于分配问题结构特殊，可行域是**双随机矩阵**（每行每列和为 1 的 0-1 矩阵）的顶点集合，恰好是**置换矩阵**，这保证了匈牙利算法给出的解一定是整数解，无需分支定界。

3.5 实例

最简单的分配问题，2 个人分配 2 项工作，工作需时（小时）矩阵为

$$\begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix} = \begin{pmatrix} 2 & 3 \\ 3 & 2 \end{pmatrix} \quad (17)$$

答案显然：第一个人做第一项工作，第二个人做第二项工作，共需要时间 4 小时；否则（交叉分配）共需要 6 小时。

对 $n = 2$ 的式 (16) 做改造：将二维足标改为一维足标， $x_{11} = y_1$, $x_{12} = y_2$, $x_{21} = y_3$, $x_{22} = y_4$ ，则约束

$$\begin{aligned} y_1 + y_2 &= x_{11} + x_{12} = 1, \\ y_3 + y_4 &= x_{21} + x_{22} = 1, \\ y_1 + y_3 &= x_{11} + x_{21} = 1, \\ y_2 + y_4 &= x_{12} + x_{22} = 1, \end{aligned} \quad (18)$$

所以问题可写成：

$$\begin{aligned} \min \quad & f = (2, 3, 3, 2) \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{pmatrix} \\ \text{s.t.} \quad & \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}, \quad y_i = 0, 1, \quad i = 1, 2, 3, 4 \end{aligned} \quad (19)$$

```
clear
f = [2, 3, 3, 2];
Aeq = [1 1 0 0;
       0 0 1 1;
       1 0 1 0;
       0 1 0 1];
beq = [1; 1; 1; 1];
[y, fval] = bintprog(f, [], [], Aeq, beq)

% Optimization terminated.
% y =
%      1      0      0      1
% fval =
```

% 4

即 $y_1 = x_{11} = 1$, $y_4 = x_{22} = 1$, 第一人做第一项工作, 第二人做第二项工作, 总时间 4 小时, 与直觉一致。

另一种解法: 部分求解器 (如早期 `bintprog`) 只接受不等式约束, 可以把一个等式拆成两个反向不等式, 例如:

$$y_1 + y_2 = 1 \iff \begin{cases} y_1 + y_2 \leq 1 \\ y_1 + y_2 \geq 1 \end{cases} \iff \begin{cases} y_1 + y_2 \leq 1 \\ -y_1 - y_2 \leq -1 \end{cases} \quad (20)$$

对全部 4 个等式约束都作这样的拆分, 问题等价地变成只含不等式约束的 0-1 规划:

$$\begin{aligned} \min \quad & f = (2, 3, 3, 2) \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{pmatrix} \\ \text{s.t.} \quad & \begin{pmatrix} 1 & 1 & 0 & 0 \\ -1 & -1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & -1 & -1 \\ 1 & 0 & 1 & 0 \\ -1 & 0 & -1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & -1 & 0 & -1 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{pmatrix} \leq \begin{pmatrix} 1 \\ -1 \\ 1 \\ -1 \\ 1 \\ -1 \\ 1 \\ -1 \end{pmatrix}, \quad y_i = 0, 1 \end{aligned} \quad (21)$$

```
clear
f = [2, 3, 3, 2];
A = [ 1  1  0  0;
     -1 -1  0  0;
       0  0  1  1;
       0  0 -1 -1;
       1  0  1  0;
      -1  0 -1  0;
       0  1  0  1;
       0 -1  0 -1];
b = [1; -1; 1; -1; 1; -1; 1; -1];
[y, fval] = bintprog(f, A, b, [], [])

% Optimization terminated.
% y =
```

```
%      1      0      0      1
% fval =
%      4
```

两种建模方式给出完全相同的结果，说明「等式拆两个不等式」的等价变换是正确的。

思考题 $n = 3$ 的分配问题，工作需时（小时）矩阵分别为

$$\begin{pmatrix} 6 & 4 & 5 \\ 4 & 7 & 5 \\ 6 & 5 & 4 \end{pmatrix} \quad \text{和} \quad \begin{pmatrix} 6 & 4 & 5 \\ 6 & 7 & 4 \\ 4 & 4 & 6 \end{pmatrix} \quad (22)$$

请仿照实例，将二维下标改写为一维、列出等式（或不等式）约束，并用 `bintprog`（或 `intlinprog`）求解。

3.6 例：装载问题

某运输公司有 m 辆不同的卡车， $i^{\#}$ 车的最大载重量为 b_i ，现有 n 种货物可供装载，设 $j^{\#}$ 货物每件重量为 a_j ($j = 1, 2, \dots, n$)， $i^{\#}$ 车装载 $j^{\#}$ 货物每件收费为 c_{ij} ，问怎样的装载方案使公司的收入最多？

建模：记 x_{ij} 表示 $i^{\#}$ 车装载 $j^{\#}$ 货物的件数（非负整数），

则总收费（目标函数）：
$$\sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij}。$$

$i^{\#}$ 车装载的限制（约束条件）：

$$\sum_{j=1}^n a_j x_{ij} \leq b_i, \quad i = 1, 2, \dots, m. \quad (23)$$

所以得到整数规划问题：

$$\begin{aligned} \max \quad & z = \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_{j=1}^n a_j x_{ij} \leq b_i, \quad i = 1, 2, \dots, m \\ & x_{ij} \geq 0, \text{ 整数}, \quad i = 1, 2, \dots, m, j = 1, 2, \dots, n. \end{aligned} \quad (24)$$

显然，装载问题是 **0-1 背包问题** 的推广：只要取 $m = 1$ （只有一辆车），且每种货物都只有一件（此时 x_{ij} 只能取 0, 1），就退化为 0-1 背包问题。

3.7 分支定界法的基本思想

整数规划一般通过分支定界法求解，核心是「放松—分支—定界—剪枝」：

1. **放松 (Relaxation)**: 先去掉整数约束, 解相应的线性规划 (LP 松弛问题), 得到下界 (对 min 问题);
2. **分支 (Branch)**: 若 LP 最优解中某个应取整数的变量 x_k 取到非整数值 $\bar{x}_k$, 则以 $x_k \leq \lfloor \bar{x}_k \rfloor$ 和 $x_k \geq \lceil \bar{x}_k \rceil$ 为附加约束, 把问题分成两支子问题;
3. **定界 (Bound)**: 每解出一个整数可行解, 就用其目标值更新上界 (对 min 问题, 当前已知最优整数解的目标值);
4. **剪枝 (Prune)**: 若某支子问题的 LP 松弛最优值已经不小于当前上界, 则该支不可能产生更优的整数解, 无需继续分支, 直接剪掉。

反复分支、定界、剪枝, 直到所有分支都被剪掉或求出整数解为止, 此时当前最优的整数解即为全局最优解。

3.8 实例: IP 问题

$$\begin{aligned}
 \min \quad & z = 7x_1 + 3x_2 + 4x_3 \\
 \text{s.t.} \quad & x_1 + 2x_2 + 3x_3 \geq 8 \\
 & 3x_1 + x_2 + x_3 \geq 5 \\
 & x_i \geq 0, \quad i = 1, 2, 3, \text{ 整数}
 \end{aligned} \tag{25}$$

相应的 LP 松弛问题 (去掉整数限制):

$$\begin{aligned}
 \min \quad & f = 7x_1 + 3x_2 + 4x_3 \\
 \text{s.t.} \quad & x_1 + 2x_2 + 3x_3 \geq 8 \\
 & 3x_1 + x_2 + x_3 \geq 5 \\
 & x_i \geq 0, \quad i = 1, 2, 3
 \end{aligned} \tag{26}$$

用 `linprog()` 解此 LP 问题, 得

$$x^0 = (0.4, 3.8, 0), \quad f_0 = 14.2. \tag{27}$$

$x_2 = 3.8$ 非整数, 对它分支: 将 LP 分成 **LP11**: LP + “ $x_2 \leq 3$ ”; **LP12**: LP + “ $x_2 \geq 4$ ” 两部分, 并以 $f_0 = 14.2$ 作为全局下界。

解 **LP11**: $x^{11} = (0.5, 3, 0.5)$, $f_{11} = 14.5$;

解 **LP12**: $x^{12} = (0.3333, 4, 0)$, $f_{12} = 14.33$;

两支的最优解都还含有非整数分量 (x_1), 继续分支:

将 **LP11** 按 x_1 分成 **LP211**: LP11 + “ $x_1 = 0$ ”; **LP212**: LP11 + “ $x_1 \geq 1$ ”;

将 **LP12** 按 x_1 分成 **LP213**: LP12 + “ $x_1 = 0$ ”; **LP214**: LP12 + “ $x_1 \geq 1$ ”。

解 **LP211**: $x^{211} = (0, 3, 2)$, $f_{211} = 17$;

解 **LP212**: $x^{212} = (1, 0, 2.3333)$, $f_{212} = 16.33$;

解 **LP213**: $x^{213} = (0, 5, 0)$, $f_{213} = 15$;

解 **LP214**: $x^{214} = (1, 4, 0)$, $f_{214} = 19$ 。

$x^{213} = (0, 5, 0)$ 与 $x^{214} = (1, 4, 0)$ 都是整数解，是原 IP 问题的可行解，取其中较小者置上界 $\bar{f} = f_{213} = 15$ 。因为 $f_{211} = 17 > \bar{f}$ 、 $f_{212} = 16.33 > \bar{f}$ ，LP211、LP212 已不可能产生比 15 更优的整数解，剪枝；又 $f_{214} = 19 > \bar{f}$ ，该整数解也不是最优的。于是得到 IP 问题的最优解

$$x^* = x^{213} = (0, 5, 0), \quad f^* = f_{213} = 15. \quad (28)$$

分支定界流程图：

$$\begin{array}{ccc}
 & \text{IP} \Rightarrow \text{LP: } ((0.4, 3.8, 0), 14.2) & \\
 & \swarrow \quad \searrow & \\
 \begin{array}{c} x_2 \leq 3 \checkmark \\ \text{LP11} \\ ((0.5, 3, 0.5), 14.5) \end{array} & & \begin{array}{c} \swarrow \quad \searrow x_2 \geq 4 \\ \text{LP12} \\ ((0.3333, 4, 0), 14.33) \end{array} \\
 \begin{array}{c} x_1 = 0 \checkmark \quad \swarrow \quad \searrow x_1 \geq 1 \\ \text{LP211} \quad \text{LP212} \\ ((0, 3, 2), 17) \quad ((1, 0, 2.33), 16.33) \end{array} & & \begin{array}{c} \swarrow \quad \searrow x_1 \geq 1 \\ \text{LP213} \quad \text{LP214} \\ ((0, 5, 0), 15)^* \quad ((1, 4, 0), 19) \end{array} \\
 & & (29)
 \end{array}$$

（标 * 者为最终最优整数解）

3.9 例：选址问题

某地有 m 座煤矿， $i^{\#}$ 矿每年生产 a_i 吨煤，原有火力发电厂一个，每年需用煤 b_0 吨，每年运行的固定费用（包括折旧费，不包括煤的运费）为 h_0 元。现要规划新建一个发电厂， m 座煤矿每年开采的原煤全部供给这两个发电厂，现有 n 个备选厂址，若在 $j^{\#}$ 备选厂址建发电厂，每年运行的固定费用为 h_j ($j = 1, 2, \dots, n$)，每吨原煤从 $i^{\#}$ 矿运到 $j^{\#}$ 备选厂址的运费为 c_{ij} ，每吨原煤从 $i^{\#}$ 矿运到原有厂的运费为 c_{i0} 。问：新厂厂址应选在何处， m 座煤矿开采的原煤应如何分配给两个发电厂，才能使每年的总费用（电厂运行的固定费用及原煤运费之和）最小？

建模： m 座煤矿每年的总产量是 $\sum_{i=1}^m a_i$ ，原有厂每年用煤 b_0 ，故新建厂每年可用煤

$$b = \sum_{i=1}^m a_i - b_0. \quad (30)$$

令决策变量：

$$y_j = \begin{cases} 1 & j^{\#} \text{ 备选厂址被选中建新厂} \\ 0 & \text{否则} \end{cases}, \quad j = 1, 2, \dots, n. \quad (31)$$

记原有厂为 $0^{\#}$ 厂，被选中建新厂的备选厂址为 $j_0^{\#}$ 厂，即最终 $y_{j_0} = 1$ 、其余 $y_j = 0$ 。

设决策变量 x_{ij} 为每年从 $i^{\#}$ 矿运到 $j^{\#}$ 厂的原煤数量 ($i = 1, 2, \dots, m$; $j = 0, 1, \dots, n$)，则每年总费用（目标函数）为

$$\underbrace{\sum_{i=1}^m \sum_{j=0}^n c_{ij} x_{ij}}_{\text{原煤运费}} + \underbrace{\sum_{j=1}^n h_j y_j}_{\text{新建厂运行费}} + \underbrace{h_0}_{\text{原有厂运行费}}. \quad (32)$$

约束条件:

1. 对 $i^{\#}$ 煤矿来说, 每年生产的 a_i 吨煤全部运往原有厂与新建厂:

$$\sum_{j=0}^n x_{ij} = a_i, \quad i = 1, 2, \dots, m; \quad (33)$$

2. m 座煤矿运到原有厂的总煤量恰为原有厂的年用煤量:

$$\sum_{i=1}^m x_{i0} = b_0; \quad (34)$$

3. 若 $j^{\#}$ 备选厂址被选中 ($y_j = 1$), 则 m 座煤矿运到该厂的总煤量应恰为新厂年用煤量 b ; 若未被选中 ($y_j = 0$), 则运量应为 0。这两种情形可统一写成:

$$\sum_{i=1}^m x_{ij} = b \cdot y_j, \quad j = 1, 2, \dots, n; \quad (35)$$

4. n 个备选厂址中只能选一个建厂:

$$\sum_{j=1}^n y_j = 1; \quad (36)$$

5. 运量非负、选址变量为 0-1:

$$x_{ij} \geq 0 \quad (i = 1, \dots, m; j = 0, 1, \dots, n), \quad y_j = 0, 1 \quad (j = 1, \dots, n). \quad (37)$$

归纳后得到下面的规划问题:

$$\begin{aligned} \min \quad & z = \sum_{i=1}^m \sum_{j=0}^n c_{ij} x_{ij} + \sum_{j=1}^n h_j y_j + h_0 \\ \text{s.t.} \quad & \sum_{j=0}^n x_{ij} = a_i, \quad i = 1, 2, \dots, m, \\ & \sum_{i=1}^m x_{i0} = b_0, \\ & \sum_{i=1}^m x_{ij} = b \cdot y_j, \quad j = 1, 2, \dots, n, \\ & \sum_{j=1}^n y_j = 1, \\ & x_{ij} \geq 0, \quad i = 1, 2, \dots, m; j = 0, 1, 2, \dots, n, \\ & y_j = 0, 1, \quad j = 1, 2, \dots, n, \end{aligned} \quad (38)$$

注：由最优解得出某个 $y_{j_0} = 1$ ，该下标 j_0 就是要选中的备选厂址，即在 $j_0^\#$ 备选厂址建新厂；解出的 x_{i0} 、 x_{ij_0} 就是各煤矿分别运到原有电厂和新建电厂的煤量。

约束 3 中的 $b \cdot y_j$ 是决策变量 y_j (0-1 变量) 与常数 b 的乘积，仍是关于 (x, y) 的线性表达式，所以整个模型仍是 (混合) 线性约束；又因为 x_{ij} 取实数、 y_j 取 0-1 整数，两类变量并存，所以 (38) 是混合整数规划问题 (Mixed Integer Programming, MIP)。

3.10 例：旅行售货员问题 (Travelling Salesman Problem, TSP)

在城市 v_1 的一位旅行售货员计划去城市 $v_2, v_3, \dots, v_n$ 推销商品，最后回到 v_1 ，设 c_{ij} 为从 v_i 到 v_j 的距离 ($i, j = 1, \dots, n$, $c_{ii} = 0$, $c_{ij} = c_{ji}$)。问：应该怎样规划路线，使得对每个城市恰好访问一次，且总距离最短？

注 1：若将距离换成旅行时间或费用，一般不再对称，即 $c_{ij} \neq c_{ji}$ ，模型仍然适用 (此时为「非对称 TSP」)。

注 2：若原路倒走算两条不同路线，由乘法原理，全部可能路线数为 $(n-1)!$ (固定起点 v_1 ，其余 $n-1$ 个城市全排列)。

建模：设决策变量

$$x_{ij} = \begin{cases} 1 & \text{旅行路线中有从 } v_i \text{ 到 } v_j \text{ 的行程} \\ 0 & \text{否则} \end{cases} \quad (39)$$

则总距离 (目标函数)： $\sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij}$ 。

显然应有 $x_{ii} = 0$ (不能原地停留)，实现上常取 $c_{ii} = A$ (充分大的常数，即所谓「大 M 法」) 作为惩罚，使得优化过程自动避开 $x_{ii} = 1$ ，同时保持系数矩阵 $C = (c_{ij})$ 是满秩、无特殊结构的方阵，便于统一编程；更严谨的做法是直接令自变量的上界 $\text{ub}(ii \text{ 下标}) = 0$ ，从变量层面禁止自环。

对任一城市 v_i ，在最优巡回路线中恰有一次为某段行程的起点、恰有一次为终点，即

$$\sum_{j=1}^n x_{ij} = 1, \quad i = 1, 2, \dots, n; \quad \sum_{i=1}^n x_{ij} = 1, \quad j = 1, 2, \dots, n. \quad (40)$$

但仅有以上两组约束还不够——它们只保证每个顶点的入度、出度都是 1，满足这两组约束的解可能是若干个互不相交的小回路 (子回路) 拼在一起，而不是经过全部 n 个城市的一条大回路。因此还需要消除子回路 (subtour elimination) 的约束。

消除子回路方法一 (割集不等式)：要求任意把 n 座城市分成两个非空组后，两组之间一定有路线相连 (否则就说明存在一个不含全部城市的孤立子回路)。设 Q 是 $\{1, 2, \dots, n\}$ 的任一非空真子集， $\bar{Q} = \{1, \dots, n\} \setminus Q$ ，则

$$\sum_{i \in Q} \sum_{j \in \bar{Q}} x_{ij} \geq 1. \quad (41)$$

$\{1, 2, \dots, n\}$ 的非空真子集共有

$$C_n^1 + C_n^2 + \dots + C_n^{n-1} = 2^n - 2 \quad (42)$$

个，但由对称性（ Q 与 $\bar{Q}$ 产生的是同一族约束中互补的两条），实际独立约束数约为 $2^{n-1} - 1$ 量级，仍随 n 指数增长。

所以旅行售货员问题的完整模型为：

$$\begin{aligned} \min \quad & z = \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_{j=1}^n x_{ij} = 1, \quad i = 1, 2, \dots, n \\ & \sum_{i=1}^n x_{ij} = 1, \quad j = 1, 2, \dots, n \\ & \sum_{i \in Q} \sum_{j \in \bar{Q}} x_{ij} \geq 1, \quad \forall Q \subset \{1, 2, \dots, n\}, Q \neq \emptyset, \bar{Q} = \{1, 2, \dots, n\} \setminus Q \\ & x_{ij} = 0, 1, \quad i = 1, 2, \dots, n, j = 1, 2, \dots, n. \end{aligned} \quad (43)$$

消除子回路方法二（MTZ 型/势函数不等式）：记城市集合 $V = \{v_1, \dots, v_n\}$, $|V| = n$ 。对 V 的任意非空真子集 S ($2 \leq |S| \leq n-1$)，要求 S 内部使用的边数不超过 $|S| - 1$ （否则 S 内部就形成了闭合回路）：

$$\sum_{i,j \in S} x_{ij} \leq |S| - 1, \quad \forall S \subset V, 2 \leq |S| \leq n-1. \quad (44)$$

直观地说，就是在 n 个城市中任意取出 2 到 $n-1$ 个城市组成的子集，其内部不允许出现回路——这与方法一的割集不等式是等价的消子回路手段，实践中两者约束数目都随 n 指数增长，因此大规模 TSP 通常采用**割平面法**：先只加入部分约束求解，检测到子回路后再补充相应的不等式，逐步收紧。

TSP 算例

最简单的情形： $n = 4$ ，4 个城市顺时针排成矩形，相互距离符合勾股数 3, 4, 5：

	1	2	3	4
1	A	3	5	4
2	3	A	4	5
3	5	4	A	3
4	4	5	3	A

(45)

其中 $A = 100$ （惩罚自环）。不计方向，一共有 $(4 - 1)!/2 = 3$ 条本质不同的巡回路线（把每条路线和它的倒走算作同一条）；若区分方向则有 $(4 - 1)! = 6$ 条。两个显然的最优解互为倒走： $v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow v_4 \rightarrow v_1$ 与 $v_1 \rightarrow v_4 \rightarrow v_3 \rightarrow v_2 \rightarrow v_1$ 。

完整 Matlab 代码：

```
%% TSP 问题 (n = 4)
clear
% 目标函数：距离矩阵按行展开
f = [100, 3, 5, 4, ...
     3, 100, 4, 5, ...
     5, 4, 100, 3, ...
     4, 5, 3, 100];

% 割集（子回路消除）不等式约束，原为 ">= 1" 形式，
% bintprog/intlinprog 要求 "A*y <= b" 形式，故整体取负
A_ge1 = [
0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0;
0 0 0 0 1 0 1 1 0 0 0 0 0 0 0 0;
0 0 0 0 0 0 0 0 1 1 0 1 0 0 0 0;
0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0;
0 0 1 1 0 0 1 1 0 0 0 0 0 0 0 0;
0 1 0 1 0 0 0 0 0 1 0 1 0 0 0 0;
0 1 1 0 0 0 0 0 0 0 0 0 1 1 0 0;
0 0 0 0 1 0 0 1 0 0 0 1 0 0 1 0;
0 0 0 0 0 1 0 1 0 0 1 0 0 0 0 0;
0 0 0 0 0 1 0 0 0 0 0 1 0 0 1 0;
0 0 0 0 0 0 0 0 1 0 0 0 1 0 1 0;
0 0 0 0 0 0 1 0 0 0 0 1 0 1 0 0;
0 0 0 0 0 1 0 0 0 1 0 0 0 0 1 0;
0 0 0 0 0 1 0 0 1 0 0 0 0 1 0 0];
b_ge1 = ones(14,1);
A = -A_ge1;
b = -b_ge1;

% 每行、每列恰一次的等式约束
Aeq = [1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0;
       0 0 0 0 1 1 1 1 0 0 0 0 0 0 0 0;
       0 0 0 0 0 0 0 0 1 1 1 1 0 0 0 0;
       0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1;
       1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0];
```

```

        0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0;
        0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0;
        0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1];
beq = ones(8,1);

% 禁止自环：对角元素 y1,y6,y11,y16 上界设为 0
lb = zeros(16,1);
ub = ones(16,1);
ub([1 6 11 16]) = 0;

intcon = 1:16;
[y, fval] = intlinprog(f, intcon, A, b, Aeq, beq, lb, ub)

```

手工验证：对 $n = 4$ 只有 3 条本质不同回路，逐一计算总距离：

- $v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow v_4 \rightarrow v_1$: $c_{12} + c_{23} + c_{34} + c_{41} = 3 + 4 + 3 + 4 = 14$
- $v_1 \rightarrow v_2 \rightarrow v_4 \rightarrow v_3 \rightarrow v_1$: $c_{12} + c_{24} + c_{43} + c_{31} = 3 + 5 + 3 + 5 = 16$
- $v_1 \rightarrow v_3 \rightarrow v_2 \rightarrow v_4 \rightarrow v_1$: $c_{13} + c_{32} + c_{24} + c_{41} = 5 + 4 + 5 + 4 = 18$

最短路线为 14，对应 $v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow v_4 \rightarrow v_1$ （即 $x_{12} = x_{23} = x_{34} = x_{41} = 1$ ，其余 0），与直觉判断一致，且求得的最优值恰好是矩形周长 $3 + 4 + 3 + 4 = 14$ ，验证了模型与约束的正确性。

4 非线性规划（Nonlinear Programming, NLP）

4.1 一般形式与最优性条件

非线性规划的一般形式为

$$\begin{aligned}
 \min \quad & f(\vec{x}) \\
 \text{s.t.} \quad & g_i(\vec{x}) \leq 0, \quad i = 1, \dots, p \\
 & h_k(\vec{x}) = 0, \quad k = 1, \dots, q
 \end{aligned} \tag{46}$$

其中目标函数 f 或约束函数 g_i, h_k 中至少有一个是非线性的。与线性规划不同，非线性规划的最优解一般不在可行域的顶点取得，可能出现在可行域内部或边界的任意位置，且可能存在局部最优而非全局最优。

无约束情形 ($p = q = 0$): 若 f 可微，极小点 $\vec{x}^*$ 处必有一阶必要条件 $\nabla f(\vec{x}^*) = \vec{0}$ ；若 f 二阶可微，进一步要求 Hesse 矩阵 $\nabla^2 f(\vec{x}^*)$ 半正定（二阶必要条件）。据此可用梯度下降法、牛顿法等迭代求解。

约束情形：引入 Lagrange 函数

$$L(\vec{x}, \vec{\mu}, \vec{\lambda}) = f(\vec{x}) + \sum_{i=1}^p \mu_i g_i(\vec{x}) + \sum_{k=1}^q \lambda_k h_k(\vec{x}), \quad (47)$$

若 $\vec{x}^*$ 是问题的 (局部) 最优解, 且约束满足一定正则性条件, 则存在乘子 $\vec{\mu}^* \geq \vec{0}$, $\vec{\lambda}^*$, 使得 **KKT** 条件成立:

$$\begin{aligned} \nabla_{\vec{x}} L(\vec{x}^*, \vec{\mu}^*, \vec{\lambda}^*) &= \vec{0} && \text{(平稳性)} \\ g_i(\vec{x}^*) &\leq 0, \quad i = 1, \dots, p && \text{(原可行性)} \\ h_k(\vec{x}^*) &= 0, \quad k = 1, \dots, q && \text{(原可行性)} \\ \mu_i^* &\geq 0, \quad i = 1, \dots, p && \text{(对偶可行性)} \\ \mu_i^* g_i(\vec{x}^*) &= 0, \quad i = 1, \dots, p && \text{(互补松弛性)} \end{aligned} \quad (48)$$

KKT 条件是非线性规划中最重要的最优性条件, 是线性规划中「检验数判别最优」以及无约束优化中「梯度为零」的自然推广; 当问题是**凸规划** (f, g_i 凸、 h_k 仿射) 时, KKT 条件不仅必要而且充分。

4.2 二次规划 (Quadratic Programming, QP)

二次规划问题是最简单的一类非线性规划问题: 目标函数是二次函数, 约束条件是线性的等式或不等式。一般形式是:

$$\begin{aligned} \min \quad & \frac{1}{2} \vec{x}^T H \vec{x} + \vec{f}^T \vec{x} \\ \text{s.t.} \quad & A \vec{x} \leq \vec{b} \\ & A_{eq} \vec{x} = \vec{b}_{eq} \\ & \vec{l} \leq \vec{x} \leq \vec{u} \end{aligned} \quad (49)$$

其中 H 是对称矩阵, 当 H 半正定时 (49) 是凸二次规划, 此时局部最优解就是全局最优解, KKT 条件是最优的充要条件。

二次规划有许多应用, 如时间调度、工作计划、设施分配等; 某些复杂的非线性规划问题还可以通过逐次二次逼近转化为一系列二次规划问题 (SQP 方法) 来求解。而最著名的应用当属**投资组合选择的 Markowitz 模型**, 该模型于 1952 年由 Harry Markowitz 提出, 他也因此于 1990 年获诺贝尔经济学奖。

投资组合问题。假设有 n 种金融项目 (证券、基金) 可供投资, 设 ξ_j 是投资在 $j^{\#}$ 金融项目上的收益率 (随机变量), $\vec{\xi} = (\xi_1, \dots, \xi_n)^T$ 是收益率向量; 设 x_j 是投资在 $j^{\#}$ 金融项目上的资金占总投资的比例 (权重), 于是

$$x_j \geq 0, \quad \sum_{j=1}^n x_j = 1, \quad (50)$$

$\vec{x} = (x_1, \dots, x_n)^T$ 是投资组合的权重向量。因此

$$\vec{x}^T \vec{\xi} = \sum_{j=1}^n x_j \xi_j \quad (51)$$

是投资组合的收益率，也是一个随机变量。记 $E\xi_j$ 为 $j^\#$ 金融项目收益率的数学期望（预期收益率），则由期望线性性，投资组合收益率的期望是

$$E(\vec{x}^T \vec{\xi}) = E\left(\sum_{j=1}^n x_j \xi_j\right) = \sum_{j=1}^n x_j E(\xi_j). \quad (52)$$

记 σ_{ij} 为 ξ_i 与 ξ_j 的协方差 $\sigma_{ij} = E[(\xi_i - E\xi_i)(\xi_j - E\xi_j)]$ ，协方差矩阵 $\Sigma = (\sigma_{ij})$ 。

Markowitz 用随机变量 $\vec{x}^T \vec{\xi}$ （投资组合收益率）的方差来度量投资风险，即

$$\begin{aligned} \sigma^2 &= D(\vec{x}^T \vec{\xi}) = E\left[(\vec{x}^T \vec{\xi} - E(\vec{x}^T \vec{\xi}))^2\right] \\ &= E\left[\left(\sum_{j=1}^n x_j (\xi_j - E\xi_j)\right)^2\right] = E\left[\left(\sum_{i=1}^n x_i (\xi_i - E\xi_i)\right) \left(\sum_{j=1}^n x_j (\xi_j - E\xi_j)\right)\right] \\ &= \sum_{i=1}^n \sum_{j=1}^n x_i x_j E[(\xi_i - E\xi_i)(\xi_j - E\xi_j)] = \sum_{i=1}^n \sum_{j=1}^n x_i \sigma_{ij} x_j \\ &= (x_1, x_2, \dots, x_n) \begin{pmatrix} \sigma_{11} & \sigma_{12} & \cdots & \sigma_{1n} \\ \sigma_{21} & \sigma_{22} & \cdots & \sigma_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{n1} & \sigma_{n2} & \cdots & \sigma_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \vec{x}^T \Sigma \vec{x}. \end{aligned} \quad (53)$$

由此得到三种常见的 Markowitz 模型：

模型 1（限定风险、最大化收益）：投资组合收益率的期望最大，同时控制风险不超过给定水平 b ：

$$\begin{aligned} \max \quad & E(\vec{x}^T \vec{\xi}) = \sum_{j=1}^n x_j E(\xi_j) \\ \text{s.t.} \quad & \sigma^2 = \vec{x}^T \Sigma \vec{x} \leq b \\ & x_j \geq 0, \quad j = 1, 2, \dots, n, \quad \sum_{j=1}^n x_j = 1 \end{aligned} \quad (54)$$

模型 2（限定收益、最小化风险，与模型 1 对偶）：要求投资组合收益率的期望不低于给定水平 r_0 ，在此前提下使风险（方差）最小：

$$\begin{aligned}
 \min \quad & \sigma^2 = \vec{x}^T \Sigma \vec{x} \\
 \text{s.t.} \quad & E(\vec{x}^T \vec{\xi}) = \sum_{j=1}^n x_j E(\xi_j) \geq r_0 \\
 & x_j \geq 0, j = 1, 2, \dots, n, \quad \sum_{j=1}^n x_j = 1
 \end{aligned} \tag{55}$$

这是一个标准的凸二次规划（目标是凸二次型 $\vec{x}^T \Sigma \vec{x}$ ，约束线性），可以直接套用 (49) 的形式：取 $H = 2\Sigma$ （因为 quadprog 目标是 $\frac{1}{2}\vec{x}^T H \vec{x}$ ，要使其等于 $\vec{x}^T \Sigma \vec{x}$ 须令 $H = 2\Sigma$ ）， $\vec{f} = \vec{0}$ 。

模型 3（加权平均模型，也称均值-方差效用模型）：用一个权衡系数 $\rho \in (0, 1)$ 把模型 1 和模型 2 的两个目标函数加权平均，转成单一目标：

$$\begin{aligned}
 \min \quad & \rho \vec{x}^T \Sigma \vec{x} - (1 - \rho) \sum_{j=1}^n x_j E(\xi_j) \\
 \text{s.t.} \quad & x_j \geq 0, j = 1, 2, \dots, n, \quad \sum_{j=1}^n x_j = 1
 \end{aligned} \tag{56}$$

ρ 越大表示投资者越厌恶风险（更看重方差项）， ρ 越小表示越看重收益；令 ρ 在 $(0, 1)$ 内取遍所有值，对应的最优解 $\vec{x}^*(\rho)$ 描出的 (σ, E) 曲线就是著名的**有效前沿 (efficient frontier)**。

Matlab 中求解二次规划的函数是：

```
[x, f_opt] = quadprog(H, f, A, b, Aeq, beq, lb, ub, x0)
```

算例（模型 2）：设有 3 种资产，预期收益率 $E\vec{\xi} = (0.10, 0.12, 0.15)^T$ ，协方差矩阵

$$\Sigma = \begin{pmatrix} 0.04 & 0.006 & 0.01 \\ 0.006 & 0.09 & 0.02 \\ 0.01 & 0.02 & 0.16 \end{pmatrix} \tag{57}$$

要求组合期望收益不低于 $r_0 = 0.12$ ，求方差最小的投资组合。代入模型 2：

$$\begin{aligned}
 \min \quad & \vec{x}^T \Sigma \vec{x} \\
 \text{s.t.} \quad & 0.10x_1 + 0.12x_2 + 0.15x_3 \geq 0.12 \\
 & x_1 + x_2 + x_3 = 1, \quad x_1, x_2, x_3 \geq 0
 \end{aligned} \tag{58}$$

对应 quadprog 的参数： $H = 2\Sigma$ ， $f = \vec{0}$ ；不等式 $A\vec{x} \leq b$ 取 $A = -(0.10, 0.12, 0.15)$ ， $b = -0.12$ （因为 quadprog 只接受 $\leq$ 形式，需要把 $\geq$ 两边同乘 -1 ）； $A_{eq} = (1, 1, 1)$ ， $b_{eq} = 1$ ； $lb = (0, 0, 0)^T$ 。

```
Sigma = [0.04 0.006 0.01; 0.006 0.09 0.02; 0.01 0.02 0.16];
E = [0.10; 0.12; 0.15];
```

```

H = 2*Sigma;
f = [0;0;0];
A = -E';
b = -0.12;
Aeq = [1 1 1];
beq = 1;
lb = [0;0;0];
[x, f_opt] = quadprog(H, f, A, b, Aeq, beq, lb)

```

用数值方法实际求解，得到最优权重与最小方差：

$$x^* \approx (0.4215, 0.2975, 0.2810)^T, \quad \sigma^{2*} \approx 0.03492, \quad \sigma^* = \sqrt{\sigma^{2*}} \approx 0.1869. \quad (59)$$

验证：期望收益 $E(\vec{x}^{*T}\vec{\xi}) = 0.10 \times 0.4215 + 0.12 \times 0.2975 + 0.15 \times 0.2810 \approx 0.12$ ，恰好压在约束边界 $r_0 = 0.12$ 上（因为高收益资产 3 的方差与协方差都较大，为了控制总风险，最优组合只把刚好达到目标收益所需的最少比例分配给它），说明收益约束是起作用（active）的，这与二次规划在凸问题下最优解落在约束边界的直觉一致。

小结

- **线性规划**：目标、约束都线性，最优解在可行域顶点取得，用单纯形法（结合 Bland 规则保证不死循环）或内点法求解，Matlab 用 `linprog`。
- **整数规划**：在线性规划基础上加上变量取整数（或 0-1）的要求，一般用分支定界法求解（对指派问题可用更高效的匈牙利算法），Matlab 早期用 `bintprog`，现推荐 `intlinprog`。
- **非线性规划**：目标或约束含非线性项，最优性条件由 KKT 条件刻画；其中二次规划（目标二次、约束线性）是最重要的特例，典型应用是 Markowitz 投资组合模型，Matlab 用 `quadprog`。

三类模型由简单到复杂层层递进，建模的核心步骤始终一致：**明确决策变量** → **写出目标函数** → **列出约束条件** → **选择合适的求解算法/工具**。